Course Code: 2165119 **Course Name:** ASD&D Lab

Class: TE **Batch:** T23 **Semester:** V

Student Name: Adit Mokashi **Roll No:** 2511136

EXPERIMENT NO. 03

Aim: - To deploy multi-service applications using Docker Compose.

Theory:

Modern applications are rarely built as a single, monolithic unit; instead, they usually consist of multiple microservices working together (e.g., a frontend web server, a backend API, and a database). While docker run is great for launching single containers, managing multiple interconnected containers manually can become complex and error-prone.

Docker Compose is a tool that solves this problem. It allows developers to define and run multi-container Docker applications.

Core Concepts of Docker Compose:

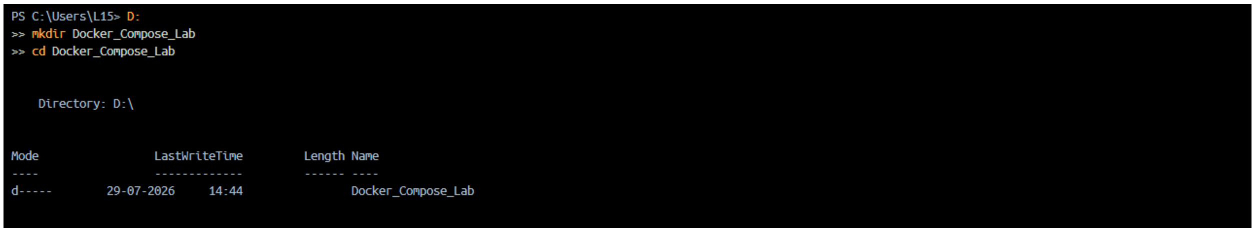
1. **YAML Configuration File (docker-compose.yml):** A human-readable text file where you define all the application's services, networks, and volumes in a structured format.
2. **Services:** These are the different containers that make up your application (e.g., web or db). Compose ensures they can communicate with each other over a shared network automatically.
3. **Single Command Orchestration:** With a single command (docker compose up), you can create and start all the services from your configuration. Similarly, docker compose down tears everything down cleanly.

By using Docker Compose, teams can ensure their development, testing, and production environments for complex applications remain consistent and easily reproducible.

IMPLEMENTATION

Step 1: Setting up the working directory

Creating a specific folder for the multi-service application.



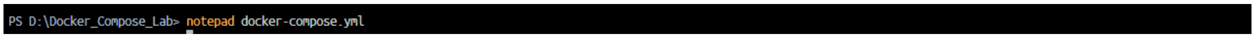
```
PS C:\Users\L15> D:
>> mkdir Docker_Compose_Lab
>> cd Docker_Compose_Lab


    Directory: D:\


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        29-07-2026     14:44                Docker_Compose_Lab
```

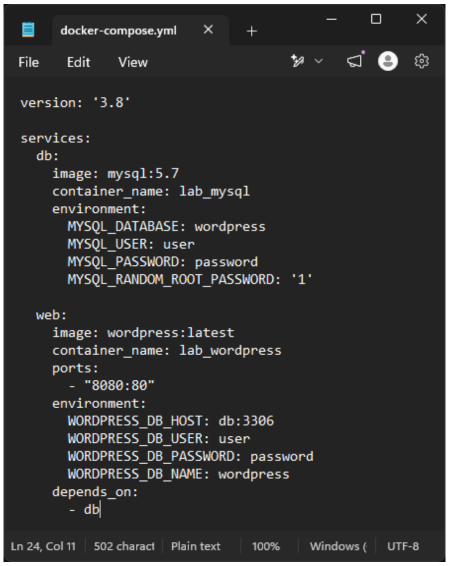
Step 2: Defining the Application Services in YAML

Creating a docker-compose.yml file using Notepad. This file defines two interconnected services: a MySQL database and a WordPress web server.



```
PS D:\Docker_Compose_Lab> notepad docker-compose.yml
```

Contents of docker-compose.yml:



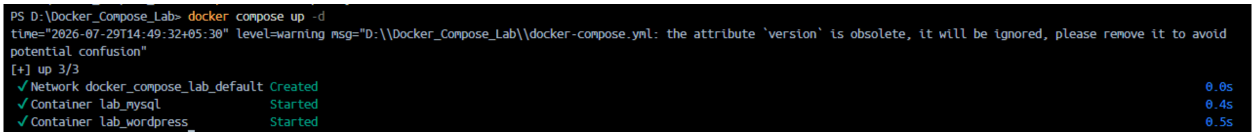
```
version: '3.8'

services:
  db:
    image: mysql:5.7
    container_name: lab_mysql
    environment:
      MYSQL_DATABASE: wordpress
      MYSQL_USER: user
      MYSQL_PASSWORD: password
      MYSQL_RANDOM_ROOT_PASSWORD: '1'

  web:
    image: wordpress:latest
    container_name: lab_wordpress
    ports:
      - "8080:80"
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: user
      WORDPRESS_DB_PASSWORD: password
      WORDPRESS_DB_NAME: wordpress
    depends_on:
      - db
```

Step 3: Deploying the Application

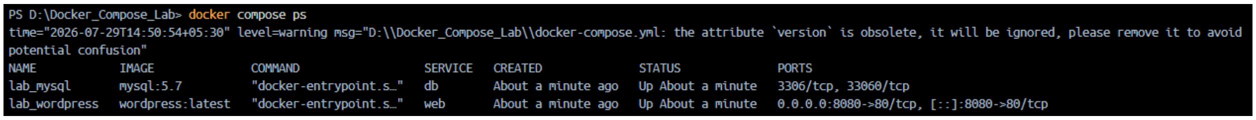
Using docker compose up -d to pull the required images, create the default network, and start all configured services in detached mode.



```
PS D:\Docker_Compose_Lab> docker compose up -d
time="2026-07-29T14:49:32+05:30" level=warning msg="D:\\Docker_Compose_Lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 3/3
 ✓Network docker_compose_lab_default Created                                                                                    0.0s
 ✓Container lab_mysql                Started                                                                                    0.4s
 ✓Container lab_wordpress            Started                                                                                    0.5s
```

Step 4: Verifying Running Services

Checking the status of the orchestrated containers.

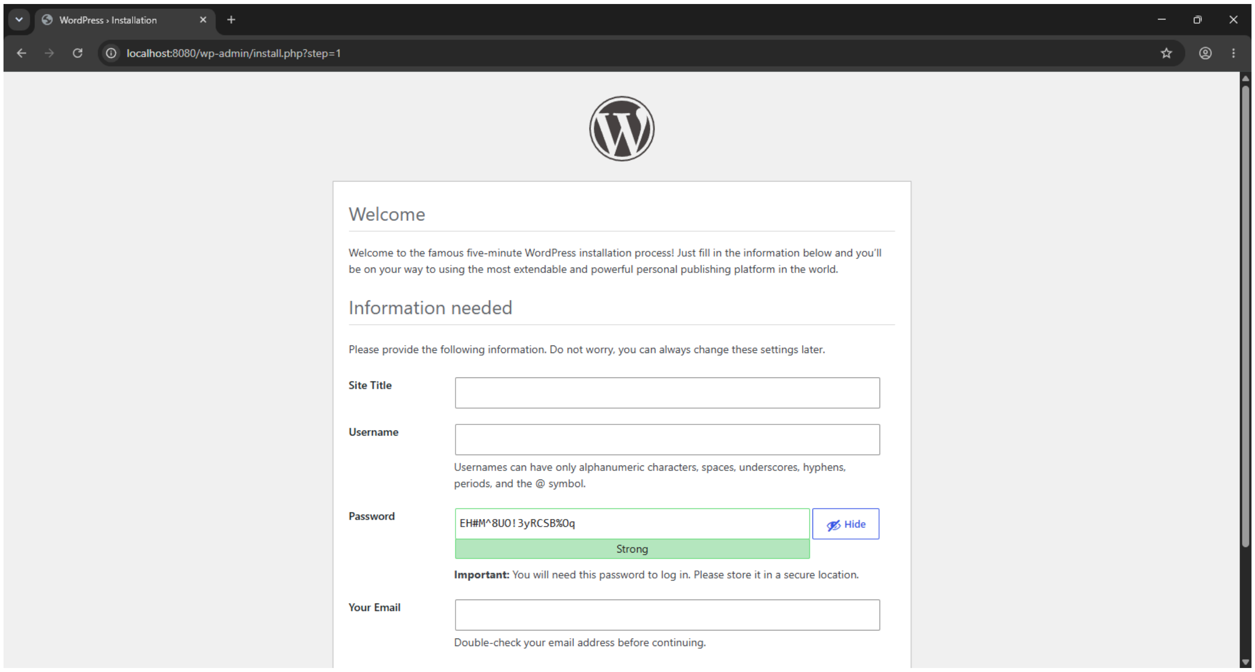


```
PS D:\Docker_Compose_Lab> docker compose ps
time="2026-07-29T14:50:54+05:30" level=warning msg="D:\\Docker_Compose_Lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid
potential confusion"
NAME            IMAGE              COMMAND                  SERVICE   CREATED              STATUS              PORTS
lab_mysql       mysql:5.7          "docker-entrypoint.s…"   db        About a minute ago   Up About a minute   3306/tcp, 33060/tcp
lab_wordpress   wordpress:latest   "docker-entrypoint.s…"   web       About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

Step 5: Testing the Application Integration

Accessing the web server via the host machine to confirm that WordPress successfully connected to the MySQL database.

Navigating to http://localhost:8080 in the web browser displays the WordPress installation screen.



Welcome

Welcome to the famous five-minute WordPress installation process! Just fill in the information below and you'll be on your way to using the most extendable and powerful personal publishing platform in the world.

Information needed

Please provide the following information. Do not worry, you can always change these settings later.

Site Title

Username

Usernames can have only alphanumeric characters, spaces, underscores, hyphens, periods, and the @ symbol.

Password EH#M^8UO!3yRCSB%Oq

Strong

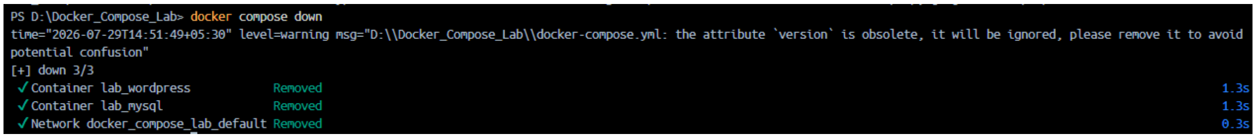
Important: You will need this password to log in. Please store it in a secure location.

Your Email

Double-check your email address before continuing.

Step 6: Tearing Down the Application (Cleanup)

Stopping and removing the containers and the custom network created by Docker Compose.



```
PS D:\Docker_Compose_Lab> docker compose down
time="2026-07-29T14:51:49+05:30" level=warning msg="D:\\Docker_Compose_Lab\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid
potential confusion"
[+] down 3/3
 ✓Container lab_wordpress           Removed                                                                                  1.3s
 ✓Container lab_mysql               Removed                                                                                  1.3s
 ✓Network docker_compose_lab_default Removed                                                                                 0.3s
```

Conclusion: Thus, we have successfully studied the concepts of multi-container orchestration and used Docker Compose to configure, deploy, test, and tear down a multi-service application consisting of interconnected web and database containers.